Computer Vision Lab

Assignment #01 | CSE480/CSE482
Ain Shams University - Mechatronics Engineering and Automation

How I approached the lab

I treated the four tasks as small experiments in how pixel values control an image. I kept the supplied images unchanged, performed every operation on NumPy arrays loaded by OpenCV, and used explicit BGR-to-RGB conversion only when displaying with Matplotlib. My aim was not only to obtain the requested pictures, but also to relate each visible change to the mask, weights, or channel assignments that produced it.

1. Selective Colour Isolation

I started by converting the apple image to grayscale because the unselected part of the final image needed brightness and texture without colour. I then separated the OpenCV image into blue, green, and red arrays. OpenCV stores them in BGR order, so naming the arrays correctly matters. A high red value alone is not enough to identify red-looking pixels: bright yellow areas can also contain a large red component. I therefore required R >= 70, R-G >= 30, R-B >= 25, and G-B <= 35. The differences were calculated with signed integers so that subtraction could not wrap around at 0 or 255. These values are image-specific thresholds: making the red differences larger would give a cleaner but smaller selection, while relaxing them would preserve more apple skin and also admit more orange or yellow background pixels.

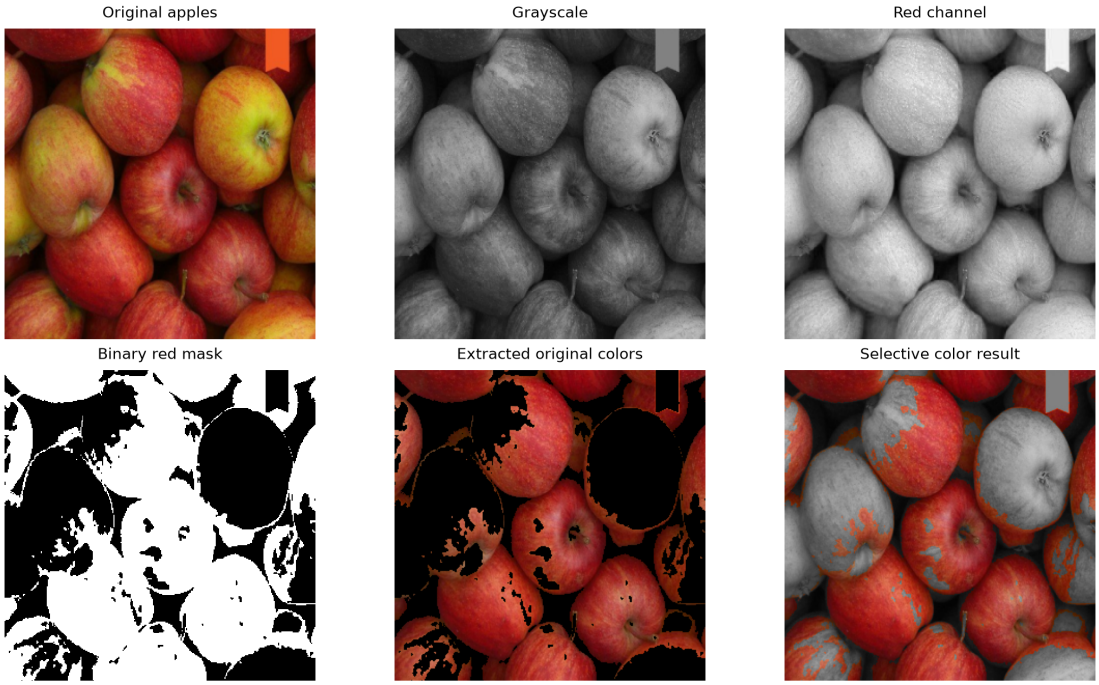


Figure 1. From left to right: the supplied image, grayscale brightness, red-channel intensity, the binary decision, the selected original pixels, and the final colour/grayscale composition.

The white regions of the mask are the pixels that passed all four tests; the black regions failed at least one test. In this result, 54.52% of the image is white. I used **bitwise_and** with the mask to keep original BGR values and used the inverted mask to take grayscale values everywhere else. Since the masks are complementary, **bitwise_or** joins the two parts without adding their intensities. Figure 1 shows why this is colour selection rather than apple detection. Red skin is retained, but yellow-green patches, highlights, stalks, and shadowed parts inside the same apples can turn gray when their channels do not meet the tests. The small red islands and gray holes follow local mottling in the fruit. The G-B condition rejects much of the orange ribbon, although a narrow red-dominant strip remains. Detecting complete apples would need spatial or object-level information, not only independent pixel tests.

2. Creating a Ghostly Overlay

Weighted addition is performed at matching array positions, so both inputs first had to have the same height, width, number of channels, and data type. The cameraman already measured 225 x 225 pixels. I resized the 425 x 313 zebra proportionally to 225 x 166, then added 29 white rows above it and 30 below it. This kept the whole zebra and avoided stretching its body. Both prepared inputs are therefore 225 x 225 uint8 BGR images. They share a canvas, but they are not geometrically registered because the subjects come from unrelated scenes.

For each output location, **cv2.addWeighted()** calculates **I = alpha C + beta Z + gamma**. I used gamma = 0 and weights that sum to 1. Alpha controls the numerical contribution from the cameraman pixel and beta controls the zebra pixel at the same coordinate. This is more specific than saying that the weights change transparency: if a cameraman pixel has intensity 80 and the corresponding zebra pixel is 240, equal weights produce about 160 after rounding.

Experiment	alpha: cameraman	beta: zebra
Cameraman dominant	0.75	0.25
Equal contribution	0.50	0.50
Zebra dominant	0.25	0.75



Figure 2. The prepared inputs and the three pixel-by-pixel blends. The relative weights change both subject visibility and the overall background level.

In the 0.75/0.25 result, the cameraman, tripod, and horizon remain the main structure, while the zebra appears as a faint second exposure. At 0.50/0.50, the zebra stripes and the cameraman silhouette have similar visual strength, so neither scene reads as a clean foreground. At 0.25/0.75, the zebra becomes dominant and the cameraman looks pale behind it. Figure 2 also becomes progressively brighter as beta increases because most of the zebra image is white; those high pixel values are blended into the entire cameraman scene, not only the animal. The ghostly appearance is therefore the superposition of two semi-weighted subjects at the same pixel locations. A cleaner composite would require separating the zebra from its background and choosing a meaningful placement, but that would be a different operation from the requested full-image weighted overlay.

3. Artistic Vignette Filtering

I wanted attention to remain on the rover and nearby terrain while the frame edges became darker. I began with a one-channel black mask and drew a white circle at the image centre. White represents a multiplication weight near 1 and black represents a weight near 0. Applying that hard circle directly would cut brightness abruptly at its boundary and leave a visible circular edge, which would look like a cut-out rather than a vignette.

To turn the boundary into a gradual transition, I blurred the circle with a Gaussian filter. The radius is 226 pixels, sigma is 91.2 pixels, and the kernel is 549 x 549. I normalized the floating-point result so the maximum central value is 1. The blur creates every intermediate weight between the bright centre and dark exterior. For each channel I then calculated **V(y,x,c) = round(I(y,x,c) x W(y,x))**. A weight of 1 preserves the source value, 0.5 halves it, and a value close to 0 makes it almost black. The same weight is applied to B, G, and R, so the intended change is brightness rather than a colour shift.

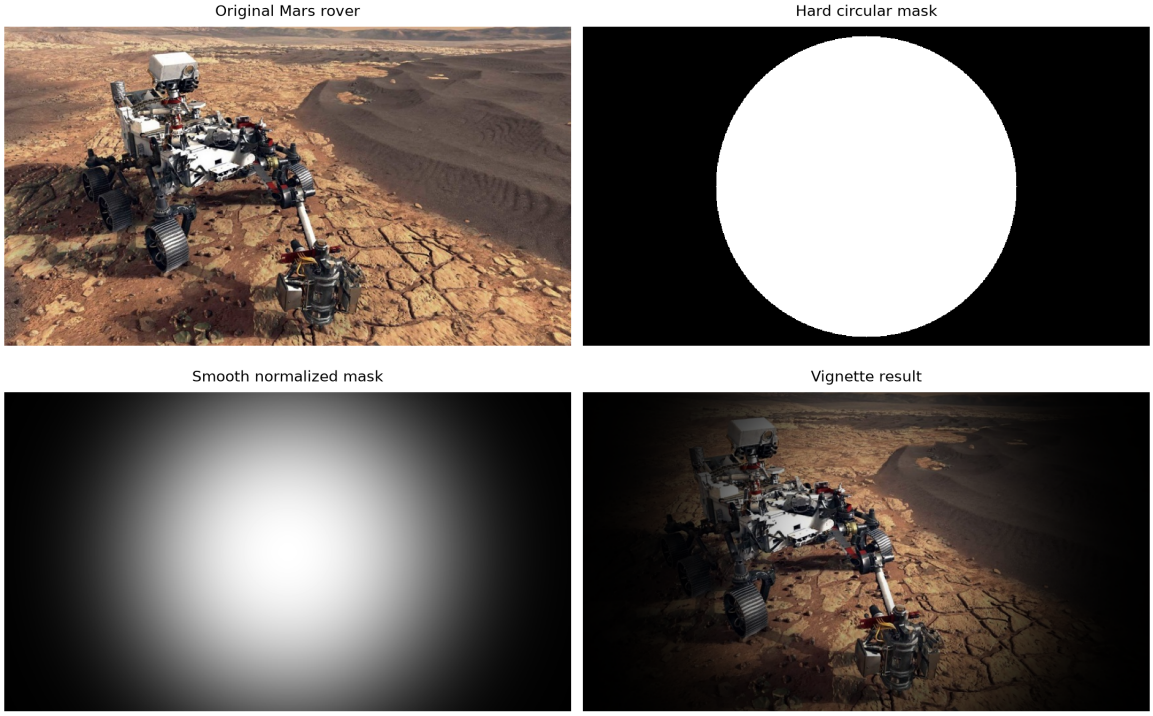


Figure 3. The hard circle defines the region of interest; Gaussian smoothing converts it into continuous weights, which darken the source progressively toward the frame edges.

Measured region	Original mean	Vignette mean
Centre quarter, grayscale	106.88	101.00
Four corner patches, grayscale	138.67	0.91

The centre pixel stays unchanged, and the centre-quarter mean changes only from 106.88 to 101.00. By comparison, the corner mean falls from 138.67 to 0.91. This matches Figure 3: the rover body, wheels, arm, and central rock texture remain readable, while the corners and distant side terrain recede. A smaller circle would darken more of the rover; a larger one would retain more of the original frame. Increasing sigma would spread the transition farther, while decreasing it would make the circular boundary more obvious. A nonzero minimum weight could preserve additional corner detail, but the current near-black corners give the strong artistic framing requested.

4. Channel Swapping - Zebra

A colour image contains three intensity values at every pixel. OpenCV stores those planes as blue, green, and red because its historical default is BGR rather than the RGB order normally used for display. Splitting the image does not create new colours; it exposes the three original intensity planes. Merging them in a new order changes which display colour receives each plane. For example, output BGR = (B,R,G) keeps blue in place, uses the original red values as green, and uses the original green values as red.

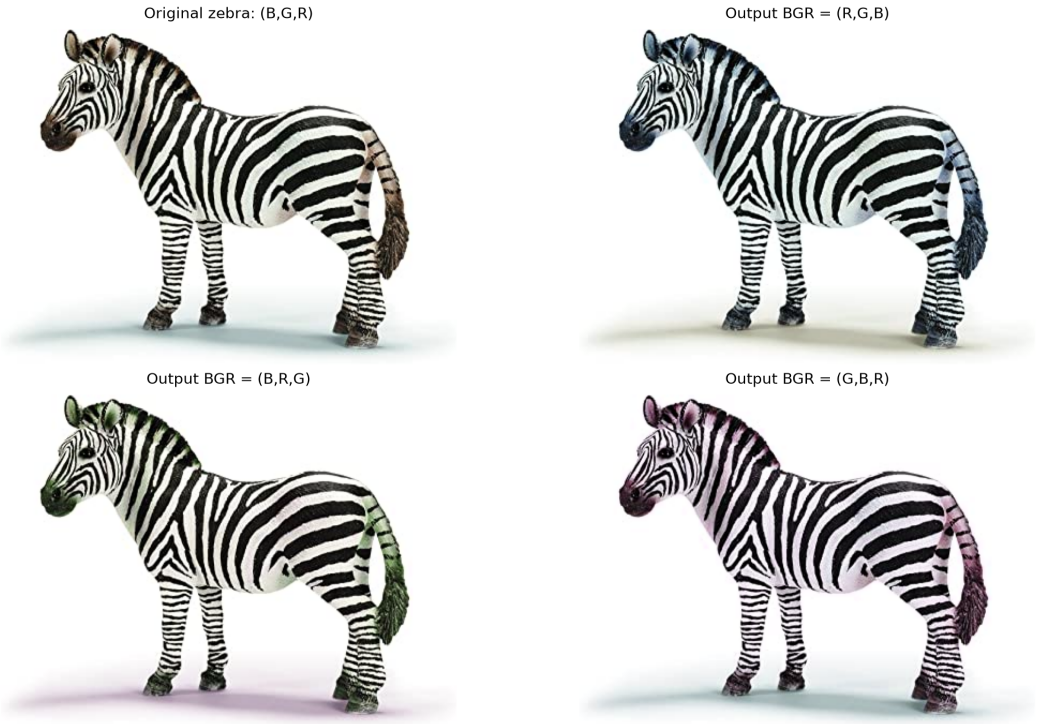


Figure 4. The original zebra and three exact channel permutations. The coat accents, muzzle, tail, and shadow change most; the neutral background and stripes change very little.

Output BGR	Mean absolute change	Maximum	Changed pixels
(R,G,B)	2.356 / 255	64	51.10%
(B,R,G)	1.408 / 255	34	45.17%
(G,B,R)	1.714 / 255	35	50.15%

The most noticeable changes in Figure 4 are the warm or cool tint around the ears and muzzle, the coloured tail, and the pale ground shadow. The black stripes, white coat, and white background remain almost the same. This happens because 46.47% of the zebra pixels are exactly neutral, meaning B=G=R; any permutation of three equal numbers returns the same pixel. Many other pixels have channels that are only slightly different, so the changed-pixel percentages are around 45-51% even though the mean change is only 1.41-2.36 intensity levels out of 255. The result is subtle for this particular source image, but it confirms an important point: channel swapping changes colour interpretation, not image geometry or brightness structure.

Channel Swapping on the Mars Image

I repeated the same three assignments on the Mars rover image as a supplementary comparison. I did not add a tint or alter contrast; each result is still made only by reordering the original B, G, and R arrays.

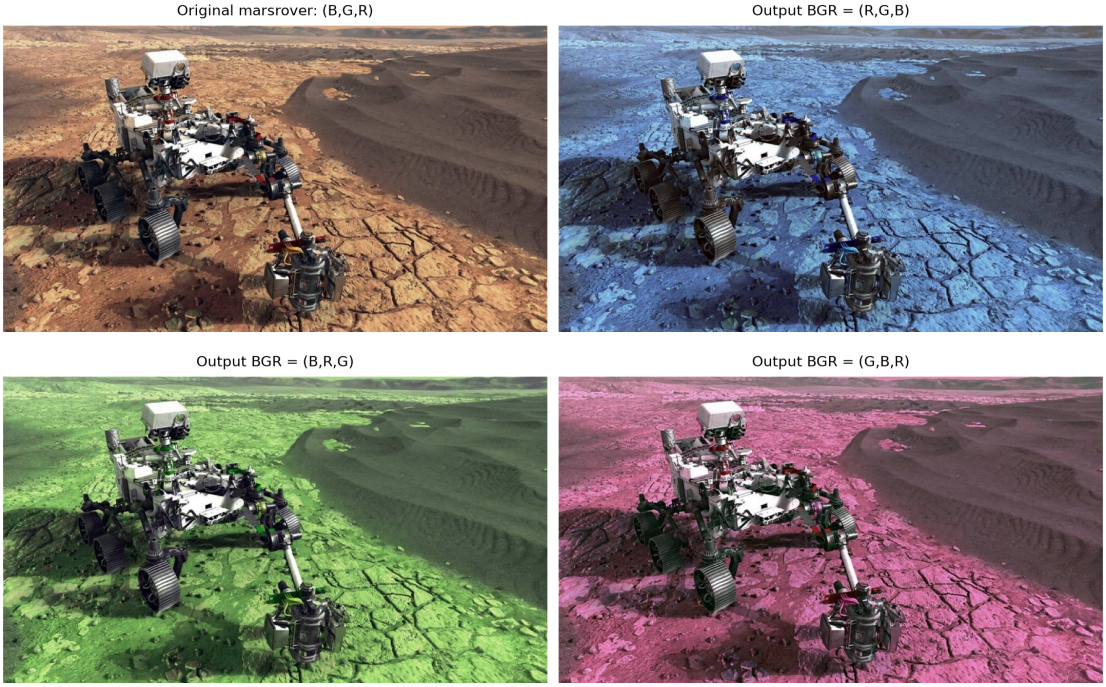


Figure 5. The Mars scene makes the same permutations easier to see because the orange-brown terrain has strongly unequal channel values.

Output BGR	Zebra mean change	Mars mean change
(R,G,B)	2.356	29.796
(B,R,G)	1.408	18.264
(G,B,R)	1.714	11.737

The Mars differences are much stronger: (R,G,B) makes the ground blue, (B,R,G) makes it green, and (G,B,R) makes it magenta. Their mean absolute changes are 29.796, 18.264, and 11.737 levels, respectively, all much larger than for the zebra. The orange terrain originally contains more red than blue, so sending those unequal values to different output channels changes the perceived colour across most of the frame. Bright neutral rover panels still remain relatively stable. Comparing Figures 4 and 5 shows that a permutation is not inherently dramatic; its appearance depends on how similar or different the source channels are at each pixel.

What I learned and what remains limited

The four results connect directly to their pixel operations. Complementary masks can combine colour and grayscale exactly, but a colour rule does not understand object boundaries. Weighted addition makes both images visible, but it also mixes their backgrounds. A blurred multiplication mask gives a controllable, smooth vignette, although its framing depends on radius and blur scale. Channel permutation preserves spatial detail while changing colour meaning, and neutral pixels expose why some images respond less than others. The executed notebook uses these same parameters and produces all five figures shown here; its assertions also confirm the four task outputs and reopen all 38 saved PNG files.